RSA-SHIELD

RSA Key Security Auditor for Legacy Critical Infrastructure

Team: [TEAM NAME]

Members: [MEMBER 1] · [MEMBER 2] · [MEMBER 3] · [MEMBER 4]

Mentor: [MENTOR NAME]

Institution: [COLLEGE / UNIVERSITY]

$$n = p \times q$$

Mathematical RSA
Key Auditing

THE PROBLEM

RSA Keys Can Fail Before Encryption Does

1. Legacy Assets

SCADA / old crypto

2. RSA Keys

Quality of key material

3. Weak Modulus

Poor entropy risk

4. Exposure

Factorization risk

Manual inventory checks cannot prove mathematical strength.

EXISTING GAP

From Key Inventory to Mathematical Assurance

Traditional Inventory

- Key inventory
- Length check
- Config check
- No math proof

RSA-SHIELD

- Structural analysis
- Mathematical analysis
- ECM attempt
- Evidence-based report

OUR SOLUTION

RSA-SHIELD: Mathematical RSA Security Auditor

Operationalizes established crypto math into a practical audit tool.

We did NOT invent RSA or ECM — we apply them for authorized auditing.

AUDIT ENGINE

Inside the Audit Engine

$n = p \times q \rightarrow$ recovering factors compromises RSA

1. Accept public key / modulus
2. Validate format
3. Extract n and e
4. Preliminary checks
5. Configurable ECM
6. Detect factors
7. Estimate exposure
8. Generate audit result

CRYPTOGRAPHIC FOUNDATION

Why Elliptic Curve Method (ECM)?

- ECM = established factorization algorithm
- Useful for finding relatively small factors
- RSA modulus: n = p × q
- Non-trivial factor = concrete weakness evidence

ECM Audit Flow

RSA modulus n

↓ ECM search

↓ Curve operations

Factor? YES → evidence

Factor? NO → inconclusive

ECM does NOT instantly factor modern strong RSA keys.

SYSTEM DESIGN

RSA-SHIELD Architecture

Frontend	React + TypeScript + Tailwind
API	FastAPI + Python
Engines	Parser · Analyzer · ECM · Risk · Reports
Data	PostgreSQL / SQLite
Output	JSON + PDF audit reports

ANALYSIS ENGINE

More Than a Factorization Tool

- L1 Format & Parameter Validation
- L2 RSA Parameter Analysis
- L3 Modulus Characteristics
- L4 ECM Factorization Attempt
- L5 Evidence-Based Risk Classification
- L6 Audit Report

Separates mathematical evidence from heuristic warnings.

AUDIT SCENARIO

From Suspicious Key to Security Evidence

Legacy Device → Key → RSA-SHIELD → Suspicious → ECM → Factor → Alert → Rotation

Mock Audit Dashboard

Key ID
LEGACY-SCADA-0042

Modulus
1024-bit

Exponent
e = 65537

ECM status
Factor discovered

Severity
CRITICAL

Action
Recommend key rotation

IMPLEMENTATION

Technology Stack

UI	React · TypeScript · Tailwind · Vite
API	Python · FastAPI · Pydantic
Engine	RSA math · Modular arithmetic · Lenstra ECM
Data	PostgreSQL / SQLite
DevOps	Docker · Git · GitHub
Testing	Pytest · Unit + Integration

PRODUCT

Key Features

- RSA key upload
- Batch auditing
- Modulus analysis
- ECM attempts
- Audit history
- Risk classification
- Evidence tracking
- JSON/PDF reports
- API access
- Audit simulation
- Visualizations
- Evidence tiers

EVALUATION

How We Measure the System

- Keys successfully parsed
- Audit processing time
- ECM execution time
- Factor discovery on weak test keys
- False-positive rate (heuristics)
- Batch throughput / resource use

Prototype Benchmark

— To Be Measured —

No fabricated numbers.

RESPONSIBLE USE

What RSA-SHIELD Does — and Does Not — Claim

Does

- Audit authorized RSA keys
- Structural + math analysis
- ECM as evidence probe
- Evidence-tiered labels
- Readable reports

Does Not Claim

- Not a magic break of modern RSA
- No factor ≠ security proof
- Not a formal proof
- Needs specialist validation
- Authorized keys only

ROADMAP

From RSA Auditor to Crypto Asset Intelligence

- Large-scale inventory
- Certificate scanning
- Weak-key patterns
- Historical monitoring
- PQC readiness
- Enterprise dashboards
- CI/CD scanning
- Compliance reports
- Hardware metadata

Know Your Keys. Prove Their Strength.

[TEAM NAME] · [MEMBERS] · [COLLEGE] · GitHub / Demo placeholders